Raspberry Pi Documentation for Core 3 Pipeline

Runs the whole pipeline on CPU 3 (except ethernet packets sent to core 0-2). All other processes run on CPUs 0-2 (desktop, wifi, ssh, etc)

To ssh: ssh straindonenew@100.97.120.117 (this is still tailscale)

PROCESS:

1. Plug in radio and mocap Ethernet (or join vicon wifi)
2. Run drone start (or drone start vicon)
3. Run drone check to make sure all is OK and /crashDrone/pose has a rate in Hz
4. Fly and when done, run drone stop

COMMANDS:

Drone start → start OptiTrack mode

Drone start vicon → start vicon mode (if optitrack running already, itll stop it)

Drone check → health check for CPU isolation, network, and message rates on each topic (will either say OK, WARN or FAIL)

Drone status → which mode is running and which CPU each thread is on

Drone logs → follow all live logs

- Drone logs natnet → OptiTrack connection: Motive found, frame rate, rigid bodies detected
- Drone logs publisher →mocap rate and flight controller send rate every 10s, time sync, commands
- Drone logs mavproxy → radio and flight controller link, mode changes, arming

Drone restart <name> → restart one service: natnet, vicon-driver, vicon-realy,mavporoxy or publisher

Drone stop → stops whatever is running

Drone enable/ drone enable vicon → start that mode automatically on boot

Drone disable → start nothing at boot

COMMANDS TO START NEW PI

1. Copy new_pi_setup.tar.gz to the new pi and unpack it
2. From inside the new_pi_setup folder run as yourself (not sudo):
    a. Bash setup.sh
    b. Wait until it prints “Setup Finished”
3. Sudo reboot
4. If need vicon wifi
    a. sudo bash ~/add_wifi.sh "ASPEN-VICON"
5. Plug in radio and ethernet then
    a. Drone start
    b. Drone check
    c. (every line should say OK)

BEFORE RUN NEW_PI_SETUP:

1. Flash SD card with Ubuntu Desktop 24.04
2. Create a username and connect to wifi
3. Enable ssh on the pi
    a. Sudo apt update && sudo apt install -y openssh-server
4. Under new_pi_setup/drone_pipeline/pipeline.conf check MOTIVE_IP, PI_MOCAP_IP, VICON_HOST, VICON_BODY, RADIO_DEV
    a. MOTIVE_IP, current=192.168.50.1 (only change if MOTIVE PC’s ethernet is set to dif address
    b. PI_MOCAP_IP, current= 192.168.50.2 (change if you want the Pi on a different address for the ethernet)(keep it in the same 192.168.50.x range as MOTIVE_IP)
    c. VICON_HOST, current=192.168.20.100:801 (change if vicon server has dif address)
    d. VICON_BODY, current = QuadPlane.QuadPlane (change if the drone is named differently in vicon)
    e. RADIO_DEV, current=/dev/ttyUSB0
    f. MAVPROXY_EXTRA_OUTS="udp:100.98.109.70:14550" (currently empty)
    g. FC_RATE_HZ = 100 (change if you want to change the output rate)
5. On Motive PC,
    a. Go to Settings → Network & internet → Ethernet, and pick the adapter the Pi's cable is plugged into.
    b. Next to **IP assignment**, click **Edit** and change it to **Manual**.
    c. Turn on **IPv4** and enter:

i. IP address: 192.168.50.1

ii. Subnet mask: 255.255.255.0

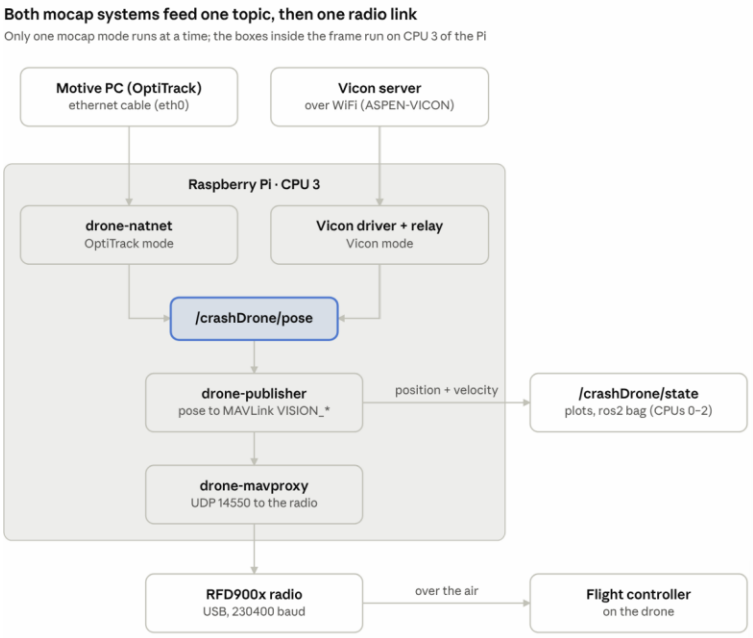
iii. Gateway and DNS: leave empty

d. Click Save.

e. In Motive set streaming → local interface to 192.168.50.1, and allow motive through windows firewalls (tick public) (kkep transmission type as unicast) *** motive streams from only on elocal interface ast a time. Once switched to 192.168.50.1, anything else that receives motives tream over wifi stops getting it

f. (run this later sudo bash ~/add_wifi.sh "ASPEN-VICON")

Mocap driver → /crashDrone/pose → position_publisher → MAVLink over UDP → MAVProxy → Radio → Flight Controller/drone



Folders:

/Documents/drone_position_hold →Where the ROS2 code is held including:

Natnet_ros2 : receives optitrack frames from motive and published to crashDrone/pose

Position_publisher : Takes pose and converts to position and velocity to send over MAVLink

Visualizer : live plotter

/drone_pipeline → installs and manages processes

Bin/drone : drone commands (start,stop,check,radio,logs,env)

Pipeline.conf : all the settings

Step by Step (HOW IT WORKS)

If each of these stages crashes, system restarts it after 2 seconds

1. Get position from mocap system
   OptiTrack mode (drone-natnet)
   - Motive PC streams the drone's position over a direct ethernet cable to the Pi's eth0 port
   - Eth0 has a fixed address (192.168.50.2) and the Motive PC is 192.168.50.1. Never carries internet, internet is on wifi
   - Natnet_ros2 node receives the stream and publishes the drone's pose on the ROS 2 topic /crashDrone/pose
   - If the cable is unplugged, the serbice waits and retries until it has its IP addr

   Vicon mode (drone-vicon-driver and drone-vicon-relay)

   - Vicon server (192.168.20.100:801) is reached over WIFI
   - Vicon driver publishes every tracked object on /rigid_bodies
   - Vicon-relay.py picks out the drone and republishes it on /crashDrone/pose (same topic as OptiTrack)
   - Relay turns on vicon driver and waits if the vicon server cannot be reached and turns the driver back on if it restarts
2. Turn position into flight-controller messages (drone-publisher)
   - Position_publisher reads /crashDrone/pose and converts it into MAVLink VISION_POSITION_ESTIMATE and VISION_SPEED_ESTIMATE messages (format that the flight controller understands)
   - Sends them to MAVProxy over a local network port (127.0.0.1:14550

    - Also publishes /crashDrone/state (position and velocity) for plotting and logging… velocity is in world frame
    - Posotions are passed on without rotation so the mocap world frame must already be set up as north east down
3. Send It over the radio (drone-mavproxy)
    - MAVProxy connects the local port 14550 to the radio plugged in at /dev/ttyUSB0 (230400 baud)
    - The radio on the drone hands message to fligiht controller
    - If radio isn't plugged in, MAVProxy waits until it appears

- The kernel is told to keep CPU 3 to itself (isolcpus) and all normal programs are given to CPUs 0-2. The pipeline lives in drone.slice which only uses CPU 3
- Each service runs with Round Robin Scheduling
- Cpu-setup.sh sends the mocap ethernets interrupts to CPU 3 and all other interrupts to CPUs 0-2. Locks CPU clock at full speed as well

To see the drone state through ssh

(in another terminal) (these run on CPUs 0-2)

eval "$(drone env)"

ros2 topic echo /crashDrone/state

ros2 bag record /crashDrone/state /crashDrone/pose

ros2 run position_publisher drone_commander

What does setup.sh do?

- Checks that ubuntu 24.04, 64 bit, 4+ cores with internet
- Removes windows line endings from the scripts, in case the folder went through windows
- Copies src folders to ~/Documents/drone_position_hold/ros2_ws and ~/Documents/drone_position_hold_Vicon/ros2_ws. Also adds drone_env.sh and add_wifi.sh to home folder
- Adds ROS 2 package source and installs ROS 2 Jazzy, MAVROS, python tools, scipy and boost

- Installs pymavlink for system python and MAVProxy in ~/mavlink_env
- Builds the optitrack workspace then vicon (a vicon build failure is a warning and optitrack will still work)
- Registers the ROS and workspace libraries in /etc/ld.so.conf.d
- Sets your username and folders In pipeline.conf then runs drone_pipeline/install.sh

What install.sh runs for you?

Command	What it does
`install -d -m 755 /etc/drone-pipeline`	Creates the settings folder
`install -m 644 pipeline.conf /etc/drone-pipeline/pipeline.conf`	Copies the settings file. If one is already there, it keeps it and only adds settings that are missing
`sed -i 's/\r$//' <file>`	Removes Windows line endings, which break bash scripts. Run on every copied file
`id $DRONE_USER`, then checks for `/opt/ros/jazzy` and the built `position_publisher`	Stops with an error if the user, ROS or the build is missing. A missing Vicon build is only a warning
`command -v mavproxy.py` (as the drone user)	Finds MAVProxy. Falls back to `~/mavlink_env/bin/mavproxy.py`, then saves the path as `MAVPROXY_BIN`
`install -d -o $DRONE_USER $FLIGHT_LOG_DIR`	Creates the flight log folder
`usermod -aG dialout $DRONE_USER`	Adds the user to the `dialout` group, which is allowed to use the radio's USB serial port

Command	What it does
`install -m 644 lib/ros-env.sh lib/vicon-relay.py /usr/local/lib/drone-pipeline/`	Copies the helper files
`install -m 755 lib/run-*.sh lib/cpu-setup.sh /usr/local/lib/drone-pipeline/`	Copies the start-up scripts and makes them executable
`install -m 755 bin/drone /usr/local/bin/drone`	Installs the `drone` command so it works from any folder
`sed -e 's/@DRONE_USER@/.../' ... systemd/<unit> > /etc/systemd/system/<unit>`	Copies each service file, filling in the username, CPU core and priorities from the settings

Command	What it does
`systemctl set-property system.slice AllowedCPUs=0-2`	Background system services may only use CPUs 0–2
`systemctl set-property user.slice AllowedCPUs=0-2`	Logged-in users' programs (terminals, desktop) may only use CPUs 0–2
`systemctl set-property init.scope AllowedCPUs=0-2`	The init process may only use CPUs 0–2
`systemctl disable --now irqbalance.service`	Turns off irqbalance, which would undo the interrupt pinning

Command	What it does
`cp /boot/firmware/cmdline.txt /boot/firmware/cmdline.txt.bak-<date>`	Backs up the kernel boot options
Adds `isolcpus=domain,managed_irq,3` to `cmdline.txt`	The kernel never schedules other work on CPU 3
Adds `rcu_nocbs=3`	Moves a kind of kernel housekeeping (RCU callbacks) off CPU 3
Adds `irqaffinity=0-2`	Hardware interrupts go to CPUs 0–2 by default

Command	What it does
`nmcli connection modify <profile> connection.autoconnect no`	Stops other wired profiles (e.g. DHCP) from grabbing `eth0`
`nmcli connection delete mocap-eth`	Removes an old `mocap-eth` profile, if any

Command	What it does
Writes `/etc/NetworkManager/system-connections/mocap-eth.nmconnection`, then `chmod 600`	New profile: fixed IP `192.168.50.2/24`, never the default route (internet stays on WiFi), IPv6 off
`nmcli connection reload`	Makes NetworkManager read the new profile
`nmcli connection up mocap-eth`	Turns it on now (or when the cable is plugged in)
`install -m 755 lib/90-drone-mocap-irq /etc/NetworkManager/dispatcher.d/`	Reruns `cpu-setup.sh` each time the cable connects, because the interrupt settings reset then

Command	What it does
`systemctl daemon-reload`	Makes systemd read the new service files
`systemctl enable --now drone-cpu-setup.service`	Runs `cpu-setup.sh` now and at every boot: CPU clock at full speed, `eth0` interrupts on CPU 3, everything else on CPUs 0–2
`systemctl restart drone-cpu-setup.service`	Runs it again so new settings apply right away
`systemctl enable drone-pipeline.target`	Only with `--enable-boot`: starts OptiTrack mode at every boot

At the end it prints whether you need to reboot.

To remove it all: `sudo bash uninstall.sh && sudo reboot`.

SETTINGS YOU CAN CHANGE...

All settings live in /etc/drone-pipeline/pipeline.conf on the Pi. After editing most settings only need drone restart; CPU and priority settings need sudo bash install.sh again and then a reboot

TROUBLESHOOTING

Symptom	Likely cause	Fix
`/crashDrone/pose: no messages` (OptiTrack)	Cable unplugged, Motive not streaming, or Windows Firewall blocking	Check the cable; in Motive check Streaming → Local Interface = `192.168.50.1`; allow Motive through the firewall
`eth0 does not have 192.168.50.2`	Ethernet link is down	Plug in the cable, or run `nmcli con up mocap-eth`
`Vicon server ... not reachable`	Pi isn't on the Vicon WiFi	Connect to ASPEN-VICON
Log says `Vicon body ... not streamed`	`VICON_BODY` doesn't match the object name in Vicon	Copy the name from the log into `VICON_BODY`, then `drone restart`
Log says `Vicon body occluded`	Cameras can't see the drone	Check markers and camera coverage
MAVProxy log says `Waiting for radio`	Radio not plugged in, or on a different port	Plug it in; check `RADIO_DEV`
`ros2 topic` in a terminal shows nothing	Terminal has different ROS settings	Run `eval "$(drone env)"` in that terminal
`kernel isolated CPUs` FAIL	Pi hasn't rebooted since install	`sudo reboot`

In another terminal run this to see input output rates:

journalctl -u drone-publisher -f | grep Stats

files-

new_pi_setup/ turns new pi into drone pi

new_pi_setup.tar.gz is the zipped version of new_pi_setup

strain-drone-backup/ the files from the old pi in case the pipeline doesn't work

straindronenew-backup-2026-10-29 is a backup of the straindronenew pi

on MOTIVE name it crashDrone